Practical-3: Study and implement various operations on Python tuples such as creation, indexing, slicing, traversal, searching, nesting, and conversion between tuples and lists.

Use a Python tuple temperatures to represent the daily temperatures recorded in a city and perform the following tasks:

1. Initialize a tuple temperatures with the temperature readings of at least 10 days.

```
temperatures = (25, 26, 27, 28, 29, 30, 31, 32, 33, 34)
```

2. Display the complete tuple.

```
print("Complete tuple:", temperatures)
```

```
Complete tuple: (25, 26, 27, 28, 29, 30, 31, 32, 33, 34)
```

3. Display the first and last temperature readings.

```
print("First temperature:", temperatures[0])
print("Last temperature:", temperatures[-1])
```

```
First temperature: 25
Last temperature: 34
```

4. Display temperature readings from index 3 to index 8 using slicing.

```
print("Temperatures from index 3 to 8:", temperatures[3:9])
```

```
Temperatures from index 3 to 8: (28, 29, 30, 31, 32, 33)
```

5. Find and display the total number of days using len().

```
print("Total number of days:", len(temperatures))
```

```
Total number of days: 10
```

6. Access and display the temperature recorded on the third day.

```
print("Temperature on the third day:", temperatures[2])
```

```
Temperature on the third day: 27
```

7. Attempt to update the temperature of a specific day and observe the error generated.

```
try:
    temperatures[2] = 30
except TypeError as e:
    print("Error:", e)
```

```
Error: 'tuple' object does not support item assignment
```

8. Convert the tuple into a list, update a temperature value, and convert it back into a tuple.

```
temperatures_list = list(temperatures)
temperatures_list[2] = 30
temperatures = tuple(temperatures_list)
print("Updated tuple:", temperatures)
```

```
Updated tuple: (25, 26, 30, 28, 29, 30, 31, 32, 33, 34)
```

9. Traverse the tuple using a for loop.

```
print("Traversing using for loop:")
for temp in temperatures:
    print(temp)
```

```
Traversing using for loop:
25
26
30
28
29
30
31
32
33
34
```

10. Traverse the tuple using a while loop.

```
print("Traversing using while loop:")
i = 0
while i < len(temperatures):
    print(temperatures[i])
    i += 1
```

```
Traversing using while loop:
25
26
30
28
29
30
31
32
33
```

```
34
```

11. Search for a given temperature and display its index position.

```
search_temp = 30
if search_temp in temperatures:
    print("Temperature", search_temp, "found at index", temperatures.index(search_temp))
else:
    print("Temperature", search_temp, "not found")
```

```
Temperature 30 found at index 2
```

12. Count and display the number of days with temperature greater than 30°C.

```
count = 0
for temp in temperatures:
    if temp > 30:
        count += 1
print("Number of days with temperature greater than 30°C:", count)
```

```
Number of days with temperature greater than 30°C: 4
```

13. Sort the temperature readings by converting the tuple into a list and store the result back as a tuple.

```
temperatures_list = list(temperatures)
temperatures_list.sort()
temperatures = tuple(temperatures_list)
print("Sorted tuple:", temperatures)
```

```
Sorted tuple: (25, 26, 28, 29, 30, 30, 31, 32, 33, 34)
```

14. Display the temperatures in ascending order.

```
print("Temperatures in ascending order:", temperatures)
```

```
Temperatures in ascending order: (25, 26, 28, 29, 30, 30, 31, 32, 33, 34)
```

15. Display the temperatures in descending order.

```
temperatures_list = list(temperatures)
temperatures_list.sort(reverse=True)
temperatures = tuple(temperatures_list)
print("Temperatures in descending order:", temperatures)
```

```
Temperatures in descending order: (34, 33, 32, 31, 30, 30, 29, 28, 26, 25)
```

16. Reverse the tuple using slicing.

```
temperatures = temperatures[::-1]
print("Reversed tuple:", temperatures)
```

```
Reversed tuple: (34, 33, 32, 31, 30, 30, 29, 28, 26, 25)
```

17. Find and display the maximum and minimum temperatures.

```
print("Maximum temperature:", max(temperatures))
print("Minimum temperature:", min(temperatures))
```

```
Maximum temperature: 34
Minimum temperature: 25
```

18. Find and display the sum and average of the temperature readings.

```
print("Sum of temperatures:", sum(temperatures))
print("Average temperature:", sum(temperatures) / len(temperatures))
```

```
Sum of temperatures: 298
Average temperature: 29.8
```

19. Create a nested tuple to store (Day, Date, Temperature) details.

```
temperature_details = (("Monday", "2024-01-01", 25), ("Tuesday", "2024-01-02", 26), ("Wednesc
```

20. Display all entries from the nested tuple.

```
print("Temperature details:", temperature_details)
```

```
Temperature details: (('Monday', '2024-01-01', 25), ('Tuesday', '2024-01-02', 26), ('Wednesday
```

21. Extract and display only the temperature values from the nested tuple.

```
print("Temperature values:", [temp[2] for temp in temperature_details])
```

```
Temperature values: [25, 26, 27]
```

22. Demonstrate tuple packing and unpacking using appropriate examples.

```
temperature = 25, 26, 27
day1, day2, day3 = temperature
print("Day 1:", day1)
print("Day 2:", day2)
print("Day 3:", day3)
```

```
Day 1: 25
Day 2: 26
Day 3: 27
```

23. Create a new tuple containing temperatures above the average temperature.

```
temperature_above_average = tuple([temp for temp in temperatures if temp > sum(temperatures) /
print("Temperatures above average:", temperature_above_average)
```

```
Temperatures above average: (34, 33, 32, 31, 30, 30)
```

24. Convert the tuple into a list, perform a modification, and convert it back into a tuple.

```
temperature_list = list(temperatures)
temperature_list[2] = 30
temperature = tuple(temperature_list)
print("Updated tuple:", temperature)
```

```
Updated tuple: (34, 33, 30, 31, 30, 30, 29, 28, 26, 25)
```

25. Compare the use of tuples and lists with respect to mutability and data safety.

```
print("\n--- Comparison of Tuples and Lists ---")

# Tuples are IMMUTABLE - cannot be changed after creation
print("\nTuples (Immutable):")
sample_tuple = (1, 2, 3)
print("Original tuple:", sample_tuple)
try:
    sample_tuple[0] = 10  # This will raise an error
except TypeError as e:
    print("Cannot modify tuple:", e)
print("Tuple remains unchanged:", sample_tuple)

# Lists are MUTABLE - can be modified after creation
print("\nLists (Mutable):")
sample_list = [1, 2, 3]
print("Original list:", sample_list)
sample_list[0] = 10  # This works fine
print("Modified list:", sample_list)

# Data Safety: Tuples are safer for data that should not change
print("\nData Safety:")
print("- Tuples are ideal for storing constant data (e.g., coordinates, database records)")
print("- Lists are ideal for data that needs to be modified (e.g., shopping cart, to-do list)"
```

```
--- Comparison of Tuples and Lists ---

Tuples (Immutable):
Original tuple: (1, 2, 3)
Cannot modify tuple: 'tuple' object does not support item assignment
Tuple remains unchanged: (1, 2, 3)

Lists (Mutable):
Original list: [1, 2, 3]
Modified list: [10, 2, 3]

Data Safety:
- Tuples are ideal for storing constant data (e.g., coordinates, database records)
- Lists are ideal for data that needs to be modified (e.g., shopping cart, to-do list)
```